

Checklist

AI-Augmented Platform Engineering

1. Identify the Right Use Case

- Start with a real business or operational problem rather than asking what can be built with AI.
- Identify high-volume, repetitive cognitive work.
- Look for opportunities involving alert duplication, documentation search, or incident triage.
- Identify tasks where pattern recognition could improve anomaly detection or event correlation.
- Identify decisions where contextual information could improve prioritization or routing.
- Avoid using AI for irreversible decisions without explicit human approval.
- Select one focused use case or runbook for the first implementation.

2. Define the Current Workflow

- Document the existing manual process.
- Identify what engineers always check first.
- Identify the logs, metrics, tickets, and documentation normally gathered.
- Separate repetitive decisions from decisions requiring significant human judgment.
- Record the current time and effort required to complete the workflow.
- Establish a baseline for current accuracy, MTTR, false positives, or other relevant metrics.

3. Select the AI Deployment Approach

- Evaluate managed APIs for reasoning-heavy or prototype workloads.
- Evaluate cloud-hosted open-source models when greater data control is required.
- Evaluate self-hosted models when local control, air-gapped operation, or high-volume cost efficiency is important.
- Consider a hybrid approach where different workloads use different model types.
- Review current licensing requirements before using or fine-tuning open-source models.
- Verify applicable data residency and compliance requirements.
- Account for model serving, routing, versioning, token metering, and latency monitoring.

4. Ground the AI with Platform Data

- Prefer RAG before considering fine-tuning.
- Identify authoritative platform documentation and runbooks.
- Include relevant incident history where appropriate.
- Include operational data such as logs, metrics, traces, or Kubernetes events where required.
- Implement document ingestion and indexing.
- Use semantic/vector retrieval for conceptually related information.
- Include keyword retrieval where exact commands or terminology matter.
- Consider hybrid retrieval combining semantic and keyword search.
- Split documents along meaningful section boundaries rather than arbitrary character limits.
- Preserve enough overlap between chunks to maintain context.

5. Define Agent Responsibilities

- Give every agent a specific and clearly bounded responsibility.
- Define what information each agent can observe.
- Define what decisions each agent can make.
- Define what actions each agent can execute.
- Define how agents communicate with one another.
- Establish an orchestrator or coordination mechanism for multi-agent workflows.
- Determine which agents can operate autonomously.
- Determine which agents require supervision.

6. Classify Actions by Risk

- Classify read-only queries and context gathering as low-risk where appropriate.
- Determine whether incident-ticket creation or documentation updates can be autonomous.
- Require human approval for actions such as service restarts, deployment rollbacks, scaling, or configuration changes.
- Require humans to decide and execute destructive or irreversible actions.
- Require human control for actions affecting backups, security policies, critical infrastructure, database migrations, or financial commitments.

7. Add Pre-Execution Safety Checks

- Dry-run AI-generated operational changes in staging or a sandbox.
- Validate generated actions against security and resource policies.
- Use policy engines such as OPA or Kyverno where applicable.
- Require explicit human approval for high-impact actions.
- Show the reviewer every proposed command or API action.
- Estimate the expected impact and blast radius.
- Define a rollback plan before execution.

8. Implement Guardrails in Code

- Enforce guardrails programmatically rather than relying only on prompts.
- Add rate limits for repeated actions or notifications.
- Apply different permissions for production and non-production environments.
- Check dependencies before performing remediation.
- Add cost thresholds that trigger approval.
- Add rollback conditions for failed or rapidly reverted actions.
- Validate inputs before including them in model prompts.
- Prevent unsanitized input from controlling agent behavior.
- Maintain strict tenant or organizational context isolation.
- Validate generated outputs before acting on them.

9. Plan for AI Failure

- Treat model failures as normal operational conditions.
- Define behavior for API outages, rate limits, or quota exhaustion.
- Use cached responses when safe and appropriate.

- Route unresolved failures to a human.
- Never silently fail or return an unexplained empty response.
- Validate response format and guardrail compliance.
- Maintain deterministic fallback behavior.
- Require human confirmation when AI triage cannot determine a reliable result.

10. Establish Confidence and Approval Thresholds

- Define confidence thresholds for autonomous actions.
- Define a middle range requiring human confirmation.
- Suppress or escalate low-confidence results rather than creating unnecessary noise.
- Combine confidence with incident severity when routing alerts.
- Recalibrate thresholds using actual operational results.

11. Build an Audit Trail

- Log every agent action.
- Record the context supplied to the model.
- Record model or agent reasoning information required by your governance process.
- Record confidence scores.
- Record whether a human approved, rejected, or modified the recommendation.
- Record the action taken.
- Record the final outcome.
- Preserve model and prompt versions needed to investigate regressions.

12. Monitor AI-Specific Metrics

- Track token usage.
- Track inference latency.
- Track confidence-score trends.
- Compare model decisions with known ground truth.
- Track cost per action.
- Track human override rate.
- Track AI-assisted versus manual MTTR.
- Track false-positive rates.
- Track developer or customer satisfaction.
- Track GPU/CPU utilization when models are hosted locally.

- Track vector database query latency.
- Track agent action queue depth.

13. Evaluate Agent Health

- Investigate consistently low confidence scores.
- Investigate high human override rates.
- Stop or restrict autonomous use if triage accuracy falls below acceptable thresholds.
- Investigate when AI-assisted resolution becomes slower than manual resolution.
- Recalibrate when false-positive rates rise.
- Investigate unexpected increases in cost or token consumption.
- Watch for unbounded agent reasoning loops.

14. Measure Business Impact

- Measure reduction in MTTR.
- Measure improvements in incident detection or prevention.
- Measure improvements in developer productivity.
- Measure reductions in alert fatigue.
- Measure triage classification accuracy.
- Measure root-cause hypothesis precision.
- Measure runbook or documentation retrieval effectiveness.
- Measure time from alert generation to ticket creation.
- Measure human approval and review time.
- Compare AI operating costs against the equivalent manual effort.
- Calculate the value of time saved.
- Confirm that the AI system creates measurable value before expanding its scope.

15. Review Before Expanding

- Confirm that guardrails work as intended.
- Confirm that failures reliably fall back to safe behavior.
- Review human overrides and rejected recommendations.
- Review model drift and accuracy trends.
- Review costs and infrastructure utilization.
- Update documentation and runbooks based on feedback.
- Expand autonomous scope only when the evidence supports it.

- Add additional agents or use cases incrementally rather than attempting a complete platform transformation at once.

Final Readiness Check

- The use case solves a measurable operational problem.
- Agent responsibilities and boundaries are clearly defined.
- Platform knowledge is grounded through reliable data or RAG.
- High-impact actions require appropriate human approval.
- Guardrails are enforced in code.
- Failure and fallback paths are tested.
- Actions and outcomes are fully auditable.
- AI-specific observability is in place.
- Success metrics and thresholds are defined.
- Business value can be measured.
- The first implementation is small enough to evaluate before expanding.